VEIN HEALTH AI / MVP TECHNICAL EDITION

Enterprise MVP PRD & Technical Plan

A patient-facing continuity-of-care platform designed around verified health information, document grounding, and deterministic safety controls.



DOCUMENT
Product Requirements Document + Technical Delivery Plan

VERSION
MVP Technical Edition

DATE
August 17, 2026

MARKET DIRECTION
Nigeria / Africa → Global

POSITIONING
Patient-facing continuity of care and health-information management

CORE THESIS
Evidence first. Verify critical fields. Escalate intelligence only when risk justifies it.

PRODUCT REQUIREMENTS • ARCHITECTURE • AI COST STRATEGY • SAFETY • DELIVERY

Contents

A structured view of product scope, architecture, AI strategy, infrastructure, delivery, safety, and release gates.

Executive Summary 5

Core decisions 5

Product Requirements and Attainability 7

Product objective 7

MVP scope 8

Features to defer 8

Functional acceptance criteria 9

Success metrics 10

Target Technical Architecture 11

Frontend stack 13

Frontend application architecture 14

Backend stack 14

AI Layer and Cost Strategy 17

The key cost decision 17

Recommended model-role architecture 17

Model comparison 18

Which primary model should actually be used? 19

The medical checker should become selective 20

Document understanding 21

Risk classifier 22

RAG retrieval 22

Deterministic safety engine 23

Cost architecture at pilot scale 24

Data, Infrastructure, Security and Interoperability 26

Longitudinal patient data model 26

Database decision 28

Infrastructure: serverless containers versus Kubernetes 28

Infrastructure options and planning budget 29

Monthly pilot run rate 30

Security architecture 31

Regulatory assumptions 32

Delivery Lifecycle, Team and Budget 33

Delivery phases 33

Calendar illustration 34

Recommended team 34

Development budget 35

What should actually be paid for first 35

Quality, Safety, Failure Modes and Release Gates 37

Failure-mode analysis 37

Medication-error prevention architecture 38

Clinical QA program 40

Automated test pyramid 41

AI change-control policy 41

Pre-launch gates 42

Final MVP recommendation 42

At a glance

The report's central delivery, cost, and safety decisions summarized visually.

10-13

weeks for a disciplined two-engineer MVP with part-time clinical/security support

$50-300

planning range per month for the pilot technical platform before heavy SMS, legal, or enterprise compliance costs

$15-50

practical monthly AI budget at the 1,000-MAU planning scenario, including headroom

0 allowed

medication actions created from unverified extraction; safety-critical fields must retain provenance

CORE ARCHITECTURE PRINCIPLE

Use deterministic software whenever a deterministic answer exists. Use source evidence wherever evidence exists. Pay for stronger models only when the risk router shows they are necessary.

Executive Summary

Vein Health is a **document-grounded continuity-of-care system**: patients upload prescriptions, laboratory results, discharge instructions, and related documents; the platform extracts structured information, asks the patient to verify safety-critical fields, explains the information in plain language, creates reminders and care actions from verified information, and maintains a longitudinal health timeline. This is consistent with the original product blueprint, which explicitly positions the product around post-consultation care rather than replacing healthcare professionals.

The central technical recommendation is to **remove the assumption that every patient interaction requires two expensive medical models**. The MVP can achieve a substantially lower AI bill by using a hierarchical architecture:

Deterministic rules → OCR/document parser → structured extraction → patient confirmation → retrieval → low-cost general model → deterministic safety validation → selective medical checker/fallback only when risk or uncertainty justifies it.

This is also more aligned with current health-AI guidance. The World Health Organization warns that generative health systems can produce false, inaccurate, biased, or incomplete statements and recommends narrowly defined tasks, appropriate evaluation, stakeholder involvement, privacy protections, and regulatory oversight rather than unbounded autonomous medical decision-making.

Core decisions

Area	MVP decision
Product scope	Continuity-of-care and document understanding, not autonomous diagnosis
Frontend	React 19.2 + TypeScript + Vite + TanStack Router + Query + Form + Table
Backend	Python + FastAPI + Pydantic + SQLAlchemy + Alembic + REST/OpenAPI
Primary database	PostgreSQL
Vector search	`pgvector` inside PostgreSQL
File storage	Encrypted S3-compatible object storage
Default AI model	Low-cost 9B-class or efficient sparse model selected by internal clinical evaluation
Expensive model strategy	Escalation/fallback only; never run a large model on every interaction
Medical checker	Optional 4B medical-native model, invoked selectively
Document processing	PaddleOCR/PaddleOCR-VL 0.9B first; general multimodal model only when needed
Risk detection	Deterministic red-flag rules + small classifier
RAG	Local 305M multilingual embeddings + PostgreSQL/pgvector
Medication safety	Source-grounded deterministic rules; no invented dosage

Area	MVP decision
Infrastructure	Serverless containers initially; no Kubernetes
Initial technical run rate	Approximately **$50-$300/month** at pilot scale, excluding legal/regulatory work and unusually heavy SMS usage
AI inference target	Approximately **$5-$30/month per 1,000 monthly active users** under the proposed narrow workload
MVP development	Approximately **10-13 weeks** with two engineering contributors plus part-time clinical/security support
Lean external development budget	Approximately **$17,500-$32,000**, excluding founder labor and regulatory/legal fees

React's official documentation currently identifies React 19.2 as the latest major/minor documentation target, while TanStack maintains current React integrations for Query, Router, and Form.

The important economic conclusion is that **model-token spend does not need to be a major MVP cost anymore**. Current managed inference pricing for suitable small and efficient models is measured in cents per million tokens to low single-digit dollars per million tokens. The primary investment should therefore go into **data correctness, safety validation, UX, clinician evaluation, privacy engineering, and auditability**, rather than paying for the largest available model.

The exact Nigerian regulatory classification of the proposed software remains **unspecified**. The company is based in Nigeria and health records are clearly treated as sensitive information by the Nigeria Data Protection Commission; Nigerian health organizations have specifically sought compliance guidance under the Nigeria Data Protection Act 2023. Whether particular future functions constitute regulated medical-device software requires a separate regulatory determination before those functions are marketed or deployed.

Product Requirements and Attainability

The original blueprint has the right long-term direction but its stated first phase contains enough features to become a much larger product than necessary. The original MVP includes document scanning, extraction, explanations, medication schedules, medication duration, reminders, symptom tracking, side-effect tracking, treatment progress, safety escalation, summaries, and a longitudinal timeline.

The recommended MVP preserves the product thesis while reducing clinical and engineering surface area.

Product objective

The MVP shall prove one primary hypothesis:

> **Patients will repeatedly use Vein Health to convert fragmented post-consultation healthcare information into verified, understandable, actionable health information.**

The system shall **not** attempt to prove that an AI system can independently diagnose patients or choose treatment.

The central patient experience is:

```
Upload
   ↓
Extract
   ↓
Verify
   ↓
Organize
   ↓
Explain
   ↓
Create Actions
   ↓
Track Over Time
```

The addition of **Verify** between extraction and organization is deliberate. A prescription OCR system confusing `0.5 mg` with `5 mg`, or `mcg` with `mg`, can create substantially greater harm than a normal document-extraction error. Generative systems in healthcare also have documented risks of inaccurate or incomplete assertions, reinforcing the need for verification rather than blind automation.

MVP scope

Capability	MVP requirement	Safety rule
Account and consent	Patient registration, login, consent version tracking	No health record before consent
Document upload	Image/PDF prescriptions, labs, discharge information	Virus/file validation; encrypted storage
Document extraction	Names, medication fields, lab values, dates, instructions	Every field linked to source
Extraction confidence	Confidence per critical field	Low-confidence fields require confirmation
Patient verification	Patient reviews critical extracted information	Mandatory for medication actions
Plain-language explanation	Explain uploaded information using source evidence	No unsupported diagnosis/ treatment
Medication schedule	Build schedule from verified prescription fields	Never infer missing dosage/ frequency
Reminders	Medication/test/follow-up reminders	Derived from verified information
Symptom tracking	Patient-entered symptom journal	Observation only; no autonomous diagnosis
Side-effect journal	Patient-entered potential side effects	Escalate concerning patterns
Health timeline	Chronological documents, medications, symptoms, actions	Provenance on every event
Consultation/document summary	Structured source-grounded summary	Clearly identify source/uncertainty
Safety escalation	Emergency/red-flag detection	Conservative escalation path
Export/share summary	Patient-readable structured summary	Explicit patient authorization

Features to defer

Several capabilities should remain outside the first production release even though they appear in the broader roadmap. This is a scope recommendation, not a rejection of the longer-term vision.

Defer from first MVP	Reason
Autonomous diagnosis	High clinical/regulatory risk
AI-generated treatment changes	Unnecessary for continuity-of-care thesis
AI-generated medication doses	High consequence of model error
Personalized therapeutic recommendations	Requires more extensive clinical validation

Defer from first MVP	Reason
Broad “health insights”	Too ambiguous for V1 acceptance criteria
Family/caregiver permissions	Additional consent/access-control complexity
Clinic dashboards	Separate B2B workflow
Patient-clinician messaging	Creates operational/clinical response obligations
Full FHIR server	Premature before external integration demand
Hospital integrations	Enterprise implementation project
Telemedicine	Different operational/regulatory product
Insurance/payment workflows	Not required to validate core hypothesis

This narrower release is materially more attainable. It concentrates AI on information transformation, summarization, extraction and communication - tasks that can be bounded by source evidence - rather than allowing the model to originate clinical decisions. WHO guidance specifically recommends designing health AI for well-defined tasks with the required accuracy and reliability and recognizes risks from automation bias and false or incomplete outputs.

Functional acceptance criteria

The most important acceptance criterion is **not** “the AI is 99% accurate.” A single global accuracy number obscures radically different levels of risk.

Instead, acceptance should be field- and workflow-specific:

Requirement	Proposed launch gate
Medication action created from unverified extraction	**0 allowed**
Missing source evidence for extracted medication fields	**0 allowed**
Model silently invents a missing dosage	**0 allowed**
Model silently changes prescription frequency	**0 allowed**
Emergency rule bypassed by conversational model	**0 allowed**
Cross-patient data access in authorization suite	**0 allowed**
Severe clinician-rated harmful responses in release gold set	**0 allowed**
Low-confidence critical OCR field automatically accepted	**0 allowed**
Every generated explanation linked internally to evidence	**Required**
Every AI run has model/prompt/evidence/version metadata	**Required**
Every structured medical datum retains provenance	**Required**

Performance targets for the pilot can be set at approximately P95 <5 seconds for ordinary text explanations and asynchronous document processing with progress feedback. These are engineering targets rather than third-party benchmark claims and should be adjusted after real Nigeria-region latency testing.

Success metrics

The product should measure the original commercial thesis - **Problem → Product → Users → Retention → Revenue** - while adding safety metrics from day one. The original blueprint already identifies repeat use, healthcare organization, care actions and continued management of the health journey as the key behavioral evidence for MVP success.

Recommended instrumentation includes:

Product: account-to-first-document activation, document verification completion, reminders created, medication schedules created, weekly active care journeys, 7/30-day retention, documents per active user, repeat upload rate.

Safety: critical-field correction rate, medication extraction correction rate, low-confidence rate, escalation rate, clinician-rated major error rate, unsupported-claim rate, emergency false-negative rate.

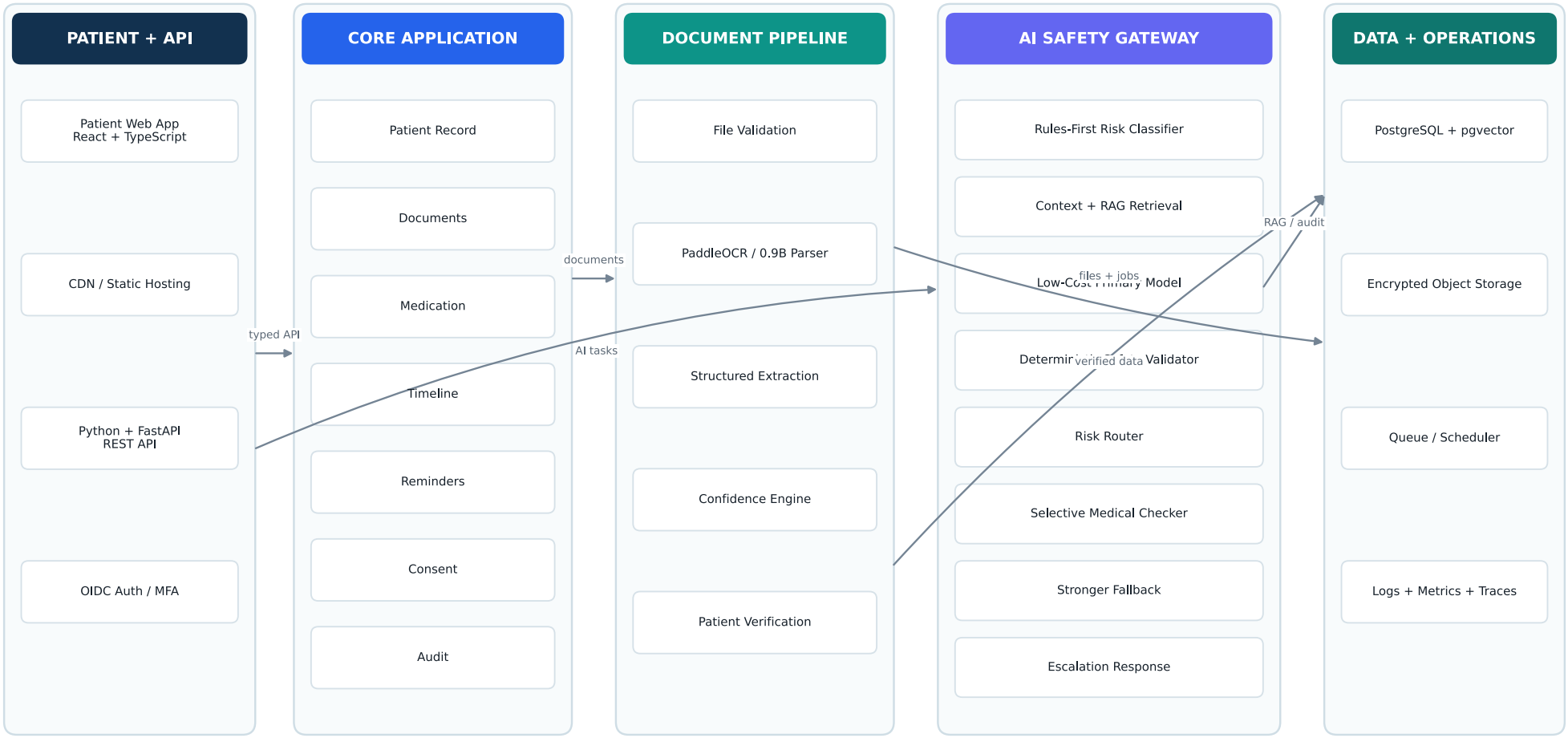
Economics: AI cost per active user, AI cost per processed document, token usage per workflow, GPU-seconds per specialist check, infrastructure cost per active user.

A reasonable engineering target is to keep **variable AI spend below $0.03 per monthly active user during the initial document-grounded product**, subject to actual usage. Current managed-model prices make that feasible under the usage assumptions later in this report.

Target Technical Architecture

The application should begin as a **FastAPI modular monolith plus asynchronous Python workers**, rather than a microservice fleet. The domain can still be separated cleanly into modules so that services can later be extracted.

That choice minimizes deployment complexity, network boundaries, observability overhead, and developer time while retaining a credible path toward enterprise architecture.



Routine path stays inexpensive; risk, ambiguity, or unsupported output triggers selective verification, fallback, or escalation.

Target technical architecture - modular monolith, document pipeline, and AI safety gateway.

Frontend stack

React 19.2 is the current documented React release. TanStack Router supports React 18/19 and strongly typed TypeScript routing; TanStack Query is the data-fetching/server-state layer; TanStack Form reached stable V1 and supports React.

Layer	Technology	Purpose
Language	TypeScript	End-to-end type safety
UI framework	React 19.2	Component architecture
Build tool	Vite	Fast SPA development/build
Routing	TanStack Router	Typed routes, route loaders, URL state
Server state	TanStack Query	API caching, mutations, retry/invalidation
Forms	TanStack Form	Typed clinical/data-entry forms
Tables	TanStack Table	Timeline/lab/document tables
Validation	Zod	Runtime schema validation
UI primitives	Radix UI	Accessible primitives
Component system	shadcn/ui	Customizable UI foundation
Styling	Tailwind CSS	Utility-based design system
Local UI state	Zustand, only where needed	Small ephemeral client state
Upload UI	Uppy or react-dropzone	Images/PDF uploads
Date handling	date-fns	Medication/reminder/date logic
Icons	Lucide	Interface icons
Unit tests	Vitest	Component/domain testing
Component testing	Testing Library	User-facing behavior
API mocking	MSW	Repeatable client integration tests
End-to-end	Playwright	Complete workflow automation

TanStack Query should own server-derived health state. A second global state system should not duplicate patient data in the browser. TanStack Query provides its own query cache and TypeScript integration.

The frontend should also avoid persistently caching sensitive health payloads in local browser storage by default. Authentication tokens should use appropriately protected session mechanisms, and service workers should cache the application shell rather than medical records.

A normal client-rendered SPA is preferable for the first release. There is no MVP need to introduce a server-component architecture, and that removes a class of complexity. React disclosed critical server-component security issues in late 2025 that required patched versions; avoiding that subsystem is a simplification, not an assertion that current patched implementations are inherently unsafe.

Frontend application architecture

```
src/
├── app/
│   ├── router/
│   ├── providers/
│   └── layouts/
├── features/
│   ├── auth/
│   ├── onboarding/
│   ├── documents/
│   ├── medications/
│   ├── reminders/
│   ├── symptoms/
│   ├── timeline/
│   ├── explanations/
│   └── safety/
├── components/
│   ├── ui/
│   └── medical/
├── api/
│   ├── client.ts
│   ├── queries/
│   └── schemas/
├── domain/
│   ├── medication.ts
│   ├── observation.ts
│   └── document.ts
├── lib/
└── tests/
```

The feature directories should encapsulate UI, data access and feature-specific validation rather than creating giant generic `components` and `services` directories.

Backend stack

The backend is **Python + FastAPI** and should remain a modular monolith with asynchronous workers for document processing, AI jobs, reminders, and other long-running work. FastAPI provides the primary REST API, Pydantic defines request/response and domain validation schemas, SQLAlchemy manages relational persistence, and Alembic manages schema migrations. This keeps the application backend aligned with the Python AI/document-processing ecosystem while preserving a clean, production-ready API boundary for the React frontend.

Layer	Recommendation
Runtime	Python 3.12+
Language	Python
Application framework	FastAPI

Layer	Recommendation
ASGI server	Uvicorn; Gunicorn/Uvicorn workers or managed container runtime in production
API approach	REST first
Contract specification	OpenAPI generated by FastAPI
Input/output validation	Pydantic
Database	PostgreSQL
Data access	SQLAlchemy 2.x + SQL for advanced queries
Schema migrations	Alembic
Vector extension	pgvector
Queue	Celery with RabbitMQ/Redis-compatible broker, or a managed cloud queue
Scheduler	Celery Beat or managed scheduler backed by durable jobs
Object storage	S3-compatible encrypted object storage
Auth	Managed OIDC/OAuth 2.0 provider with MFA
AI orchestration	Internal Python AI Gateway/service layer
ML/OCR processing	Python worker processes for OCR, extraction, embeddings, and model calls
Logging	Structured JSON + OpenTelemetry
Error monitoring	Managed error monitoring with PHI scrubbing
Metrics	OpenTelemetry → metrics backend
IaC	Terraform/OpenTofu
API tests	Pytest + FastAPI TestClient/httpx integration and contract tests
E2E	Playwright

The initial API should remain REST-based. Full GraphQL adds an additional authorization, caching and query-complexity surface that does not materially improve the core patient workflows.

Representative routes would be:

```
POST   /v1/documents
GET    /v1/documents/:id
POST   /v1/documents/:id/process
POST   /v1/extractions/:id/confirm

GET    /v1/patients/me/timeline
GET    /v1/patients/me/medications
POST   /v1/patients/me/symptoms

POST   /v1/medications/:id/reminders
POST   /v1/actions
PATCH  /v1/actions/:id

POST   /v1/ai/explain
POST   /v1/ai/summarize

GET    /v1/consents
POST   /v1/consents

GET    /v1/audit/my-access-history
```

The AI endpoint should not expose arbitrary prompts. It should accept typed **tasks** such as:

```
from typing import Literal, Union
from pydantic import BaseModel

class ExplainDocument(BaseModel):
    type: Literal["EXPLAIN_DOCUMENT"]
    document_id: str

class ExplainLabResult(BaseModel):
    type: Literal["EXPLAIN_LAB_RESULT"]
    observation_ids: list[str]

class SummarizeCareJourney(BaseModel):
    type: Literal["SUMMARIZE_CARE_JOURNEY"]
    start_date: str
    end_date: str

class ExplainMedicationInstructions(BaseModel):
    type: Literal["EXPLAIN_MEDICATION_INSTRUCTIONS"]
    medication_id: str

AIRequest = Union[
    ExplainDocument,
    ExplainLabResult,
    SummarizeCareJourney,
    ExplainMedicationInstructions,
]
```

This single design decision makes the model easier to evaluate, cheaper to prompt, safer to constrain and easier to audit.

AI Layer and Cost Strategy

The key cost decision

The system should **not** implement:

```
Every request
   → large reasoning model
   → large medical model
   → response
```

It should implement:

```
Every request
   ↓
Rules + structured data
   ↓
Can deterministic logic answer it?
   ├── Yes → no LLM
   └── No
        ↓
   Cheap model
        ↓
   Safety validator
        ↓
   Is confidence/risk acceptable?
        ├── Yes → answer
        └── No → selective checker / stronger fallback / escalation
```

This turns AI spending from a fixed “two models per turn” cost into a **risk-dependent variable cost**.

Recommended model-role architecture

Role	Recommended MVP implementation	Frequency
Primary conversational reasoning	Qwen3.5 9B **or** Mistral Small 4, decided through clinical bake-off	Routine AI interactions
Medical verification/checker	4B medical-native multimodal model	Only flagged requests
Complex fallback	Qwen3.6 27B	Small minority
Document understanding	PaddleOCR / PaddleOCR-VL 1.5 0.9B	Every uploaded document
Vision fallback	9B multimodal general model	Only OCR ambiguity
Clinical risk	Rules-first + small encoder classifier	Every applicable request
RAG embeddings	GTE multilingual 305M	Ingestion/query

Role	Recommended MVP implementation	Frequency
Reranker	Small multilingual cross-encoder	Only ambiguous retrieval
Deterministic safety	Python rules engine/service layer	Every clinical output
Medication action generation	No free-form model decision	Always deterministic

PaddleOCR's current open-source toolkit includes a 0.9B PaddleOCR-VL-1.5 model, supports structured JSON/Markdown document parsing and more than 100 languages, and is Apache-2.0 licensed. This is a much more economical first stage for prescriptions and lab documents than sending every image directly into a large general model.

Model comparison

Prices below are current public managed-inference prices as of August 2026 where published. They are normalized to **cost per 1,000 tokens**. Provider prices can change and enterprise data-handling arrangements may alter commercial terms.

Model family	Size	Best MVP role	Approx. input / 1K	Approx. output / 1K	Latency profile	Main trade-off
Qwen3.5 9B	9B	Cheapest primary candidate; multimodal fallback	**$0.00010-$0.00017**	**$0.00015-$0.00025**	Low, provider-dependent	Very cheap; general-purpose rather than medically specialized
Mistral Small 4	119B total / 6.5B active	Balanced primary candidate	**$0.00015**	**$0.00060**	Low-moderate	More expensive output, but efficient active footprint and strong general feature set
Llama 3.1 8B	8B	Cheap utility/ summarization/ classification	**$0.00005**	**$0.00008**	Very low; one service publishes ~840 tok/s	Older family; text-focused; should not be clinical reasoning default
Qwen3.6 27B	27B	Complex fallback	**$0.00060**	**$0.00300**	Fast on specialized hardware; one service publishes ~500 tok/s	~12-20× routine output price

Model family	Size	Best MVP role	Approx. input / 1K	Approx. output / 1K	Latency profile	Main trade-off
Medical-native multimodal 4B	4B	Selective clinical checker	Capacity-based if self-hosted	Capacity-based	Warm: low; cold start varies	Medical specialization, but still not reliable enough to become final authority
PaddleOCR-VL 1.5	0.9B	Document OCR/ layout parsing	Self-host/ local	Self-host/ local	Depends on CPU/GPU	Excellent cost profile, but OCR is not clinical reasoning
GTE Multilingual Base	305M	RAG embeddings	Local ≈ compute only	N/A	Very low	Retrieval model, not generator
Small encoder classifier	~100-300M	Risk classification	Local ≈ compute only	N/A	Very low	Requires a clinically labeled evaluation set

Qwen3.5 9B is currently listed as a multimodal model by managed inference providers. Two current pages from the same inference provider show slightly different prices - one catalog lists $0.10/$0.15 per million tokens, while the public pricing page lists $0.17/$0.25 - so the table deliberately gives a range instead of pretending the discrepancy does not exist.

Mistral Small 4 was released in March 2026 as a 119B-parameter sparse model with 6.5B active parameters, a 256K context, multimodal capabilities and public pricing of $0.15/M input and $0.60/M output.

For comparison, one managed service currently publishes Llama 3.1 8B at $0.05/M input and $0.08/M output with reported throughput of around 840 tokens/second; its Qwen3.6 27B endpoint is $0.60/M input and $3/M output with reported throughput around 500 tokens/second. Throughput is **not** the same as end-to-end patient latency because networking, prompt prefill, queueing and output length also contribute.

Which primary model should actually be used?

There should not be a vendor decision made solely from leaderboard scores.

The MVP should initially test two candidates:

Cost baseline: Qwen3.5 9B

Quality contender: Mistral Small 4

The two should be evaluated against the **same Vein Health clinical gold set**, containing real-world de-identified/synthetic prescriptions, lab results, discharge instructions and patient follow-up questions.

The production primary is whichever reaches all safety gates with the lowest cost.

This is important because, at expected MVP traffic, the absolute price difference is tiny.

Assume:

- 1,000 monthly active patients.
- 6 generated explanations/interactions per patient/month.
- Average 2,500 input tokens.
- Average 600 generated tokens.
- 15 million input tokens/month.
- 3.6 million output tokens/month.

Under those planning assumptions:

Primary model	Approx. model bill / 1,000 MAU / month
Qwen3.5 9B at lower listed rate	**$2.04**
Qwen3.5 9B at higher listed rate	**$3.45**
Mistral Small 4	**$4.41**
Qwen3.6 27B used for everything	**$19.80**

Those figures are arithmetic based on current listed inference prices, not vendor quotes.

The implication is significant:

> **Sacrificing clinical quality to save $2 per 1,000 patients would be false economy.**

The primary model should therefore be selected by **Vein Health's own safety evaluation**, with cost used to break ties between models that already meet the minimum clinical acceptance gates.

The medical checker should become selective

A medical-native 4B multimodal model is available and supports at least 128K context. Its official model documentation explicitly states that outputs require independent verification, and this family should therefore be treated as a specialized assistant rather than an autonomous clinical decision maker.

It should not run on every patient turn.

Trigger it only when the system sees conditions such as:

```
OCR confidence below threshold
OR
multiple medications conflict in source documents
OR
patient asks a higher-risk health question
OR
medication/allergy context produces a rule flag
OR
primary model expresses uncertainty
OR
safety validator detects unsupported clinical assertion
OR
critical field differs between extraction passes
```

A suitable routing target for the pilot is roughly:

```
70-85%  → routine low-cost path
10-20%  → specialist verification
 5-10%  → stronger-model fallback
   <5%   → explicit escalation/human review
```

Those percentages are proposed capacity targets, not clinical statistics; they should be recalibrated from pilot telemetry.

The strongest result from this approach may be discovering that the specialist checker is unnecessary for many workflows. For example:

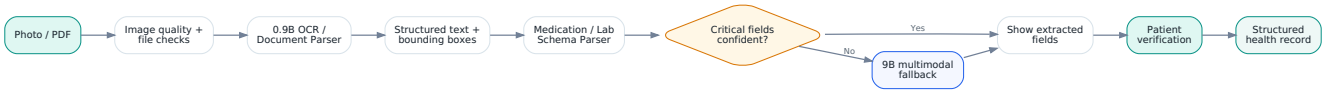
> “Your prescription says amoxicillin 500 mg three times daily for five days.”

does not need a second AI model when the source text has been extracted at high confidence, highlighted to the patient, confirmed by the patient, stored structurally, and rendered from deterministic fields.

The model becomes valuable for **explanation**, not for determining what the prescription says.

Document understanding

The cost-minimized document pipeline should be:



Cost-minimized document understanding and verification pipeline.

PaddleOCR supports PDF/image-to-structured-data conversion, more than 100 languages, and a current 0.9B document VLM designed for layout parsing. Its release materials report improvements for skew, scans, photographs, warping and lighting - particularly relevant when patients photograph paper documents rather than upload pristine PDFs.

But **OCR accuracy and clinical correctness are different problems**. The OCR system extracts visual content; the safety layer determines whether extracted content is reliable enough to become a medication or care action.

For every critical field, retain:

```
{
  "field": "dose",
  "rawText": "500 mg",
  "normalizedValue": 500,
  "unit": "mg",
  "confidence": 0.97,
  "sourceDocumentId": "...",
  "page": 1,
  "boundingBox": [0.12, 0.41, 0.31, 0.46],
  "userVerified": true
}
```

Risk classifier

Clinical risk should not be delegated entirely to a large language model.

The MVP risk system should have three layers:

```
Layer A  -  deterministic high-sensitivity rules
Layer B  -  small classifier
Layer C  -  generative model analysis
```

The deterministic layer catches known high-risk patterns and cannot be overridden by the conversational model.

The small classifier can be fine-tuned using approximately 100-300M-parameter multilingual encoder architecture. For retrieval, GTE Multilingual Base is a 305M model supporting more than 70 languages and up to 8,192 input tokens; the same general encoder class demonstrates how inexpensive local language-processing components can be compared with running a generative model.

Most importantly, the classifier should be **one-way conservative**:

> It may increase risk and trigger escalation. It should not have authority to cancel a deterministic high-risk trigger.

RAG retrieval

For the MVP, RAG should have two distinct evidence layers:

Patient evidence: verified patient documents and structured longitudinal record.

Clinical educational evidence: an approved, versioned knowledge corpus.

Patient evidence takes priority when explaining **what this patient's clinician/document says**.

External clinical information should only provide educational context and should never silently overwrite a patient's prescription or discharge instructions.

A low-cost retrieval stack is:

```
Document
   ↓
Chunk + metadata
   ↓
GTE Multilingual 305M embeddings
   ↓
PostgreSQL + pgvector
   ↓
Hybrid text/vector retrieval
   ↓
Optional reranking
   ↓
Primary generator
```

GTE Multilingual Base is a 305M-parameter model with 768-dimensional embeddings, support for over 70 languages and up to 8,192 tokens. pgvector supports exact and approximate nearest-neighbor search, HNSW and IVFFlat indexes, and hybrid use with PostgreSQL full-text search.

There is therefore **no business reason to introduce a separate vector-database product in the MVP** unless measured scale proves PostgreSQL insufficient.

Deterministic safety engine

This component costs no model tokens and may ultimately contribute more to safety than a second generative model.

Example rule categories:

Rule	Behavior
Missing dosage	Never invent one
Missing frequency	Never invent one
Missing duration	Display “not specified in source”
Low OCR confidence on dosage	Block medication schedule until confirmation
`mg` / `mcg` ambiguity	Mandatory manual confirmation
Decimal ambiguity	Mandatory manual confirmation
Conflicting source documents	Show conflict; do not choose
User asks to change prescribed dose	Decline decision; direct to clinician/pharmacist
Emergency red flag	Escalation message; no reassurance-only answer
No supporting RAG evidence	Model must express uncertainty

Rule	Behavior
Model output contains unsupported medication instruction	Block response
Source and model disagree	Source wins; flag discrepancy
Child/pregnancy/high-risk context	Increase escalation threshold
Prompt injection found inside document	Treat document text solely as untrusted data

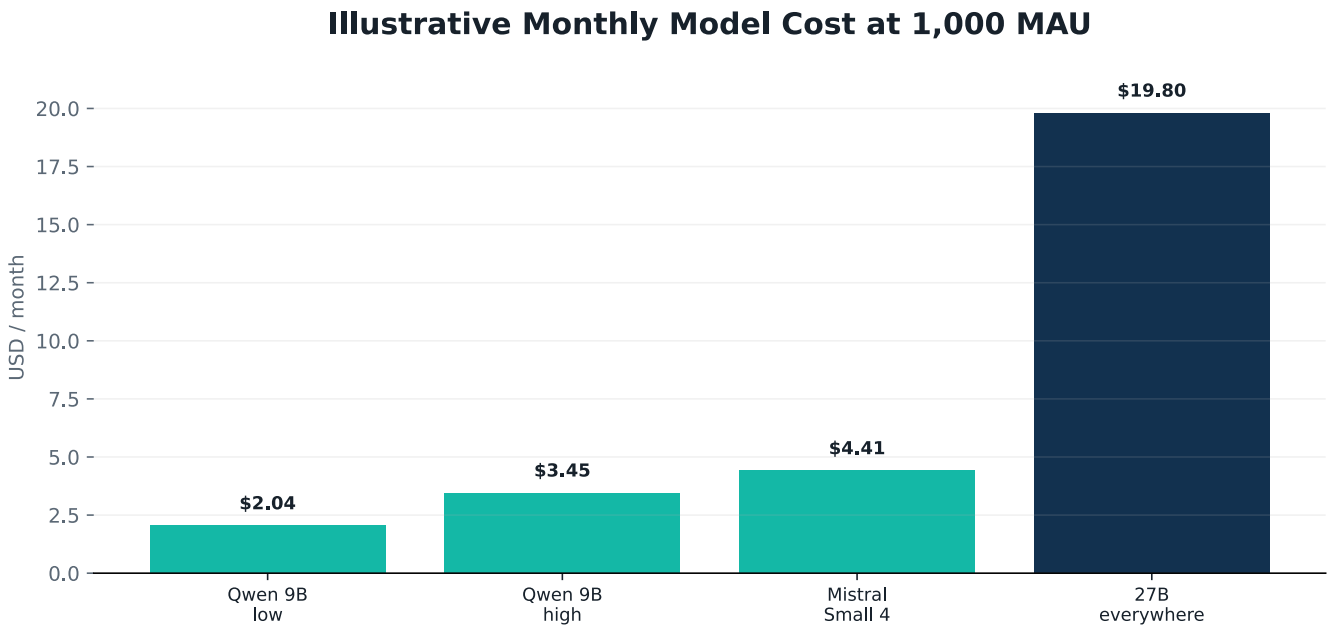
The model therefore becomes an **explanation layer inside a safety system**, not the safety system itself.

Cost architecture at pilot scale

Under the 1,000-MAU scenario above, a reasonable planning budget is:

AI function	Monthly estimate
Primary low-cost model	$2-$5
Stronger fallback	$2-$5
Selective medical checker	$5-$15
Embeddings / retrieval	<$1-$3
OCR compute	$1-$10
Evaluation / development headroom	$5-$20
Estimated practical AI budget	**$15-$50/month**

This includes deliberate headroom over pure token arithmetic. Actual costs depend heavily on images, document length, model routing, development traffic, retries, compliance-related provider contracts and the proportion of requests escalated.



Illustrative model-generation cost under the report's 1,000-MAU planning assumptions.

These bars represent **model generation only** under the defined 15M-input/3.6M-output-token planning scenario, using current public prices.

At 10,000 comparable monthly active patients, these variable-generation amounts scale approximately tenfold if usage remains linear. This remains far below the likely engineering, compliance and operational cost of the company.

That is why the recommended AI strategy is:

> **Optimize model usage aggressively, but do not optimize away safety to save a few dollars.**

Data, Infrastructure, Security and Interoperability

Longitudinal patient data model

The original blueprint correctly identifies structured health information - not uploaded documents alone - as the long-term data moat. It proposes patient profiles, documents, prescriptions, medications, laboratory results, symptoms, side effects, treatments, care plans, appointments, consultations, follow-ups and timeline events.

The database should therefore be relational and provenance-centric.

Recommended core schema:

Table	Important fields
`users`	id, auth_subject, email, status
`patients`	id, owner_user_id, tenant_id, demographics_encrypted
`consents`	patient_id, scope, policy_version, granted_at, revoked_at
`documents`	patient_id, type, object_key, checksum, source, captured_at
`document_pages`	document_id, page_number, OCR status
`extractions`	document_id, schema_version, payload, confidence
`extracted_fields`	extraction_id, field_type, raw_text, normalized_value, confidence, source_location
`medications`	patient_id, source_document_id, raw_name, normalized_name, dose, unit, route
`medication_instructions`	medication_id, frequency, duration, verified_at
`observations`	patient_id, code, value, unit, range, observed_at
`symptom_entries`	patient_id, symptom, severity, recorded_at
`side_effect_entries`	patient_id, medication_id, description, severity
`care_actions`	patient_id, type, due_at, source_id, status
`reminders`	care_action_id, channel, schedule, status
`timeline_events`	patient_id, event_type, source_id, occurred_at
`ai_runs`	task, model, model_version, prompt_version, cost, latency
`evidence_links`	ai_run_id, source_type, source_id, source_span
`safety_events`	ai_run_id, rule_id, severity, outcome
`audit_events`	actor_id, patient_id, action, resource, timestamp

Table	Important fields
`knowledge_sources`	jurisdiction, version, reviewed_at, expires_at
`knowledge_chunks`	source_id, content, embedding, metadata

A central rule should be:

> **Every health fact has an origin.**

The system should be able to answer:

```
Where did this medication dosage come from?
Who confirmed it?
When was it confirmed?
Which document/page contains it?
Which AI extraction produced it?
Which model/prompt version produced that extraction?
Has the source been superseded?
```

That level of provenance matters much more than storing a transcript of every generated sentence.

HL7 FHIR provides useful conceptual mappings. FHIR R5 is a healthcare-data-exchange standard; its DocumentReference resource represents references to documents, Observation represents measurements and point-in-time assessments, CarePlan represents intended care activities, and Provenance/AuditEvent provide concepts for origin and audit context.

The MVP should therefore be **FHIR-inspired but not a full FHIR server**.

Suggested future mapping:

Vein object	FHIR direction
Patient	Patient
Uploaded document	DocumentReference
Lab measurement	Observation
Medication	MedicationStatement / MedicationRequest depending provenance
Care journey	CarePlan
Appointment	Appointment
Follow-up action	Task
Data provenance	Provenance
Security access event	AuditEvent

That provides an integration path without forcing the engineering team to build a standards-complete interoperability platform before product-market validation.

Database decision

Recommended MVP: PostgreSQL + pgvector.

pgvector stores vector representations alongside relational data and supports exact and approximate search, HNSW/IVFFlat and hybrid strategies.

A separate vector database becomes justified only when measured production needs show one or more of:

- Vector volumes large enough to stress primary relational workloads.
- Independent vector-scaling requirements.
- Specialized filtering/replication requirements.
- Dedicated retrieval teams.
- Enterprise retrieval SLAs the relational cluster cannot satisfy.

Until then:

```
PostgreSQL
├── transactional patient data
├── structured health record
├── audit metadata
├── reminder state
├── AI provenance
├── approved knowledge chunks
└── pgvector embeddings
```

is simpler and cheaper.

Infrastructure: serverless containers versus Kubernetes

Kubernetes should not be used for the MVP.

A patient-facing pilot does not need a cluster-control plane, node pools, ingress-controller management, service meshes, autoscaler tuning, multi-cluster operations or Kubernetes-specific security policy engineering.

Use serverless containers instead:

```
Static frontend/CDN
       +
API container
       +
Worker container
       +
Managed Postgres
       +
Object storage
       +
Managed queue/cache
```

AWS's managed serverless container pricing, as one reference point, charges Fargate by requested vCPU, memory and optional storage, with no separate ECS service fee and per-second billing. Current US-East example rates show Linux/x86 CPU at $0.000011244 per vCPU-second and memory at $0.000001235 per GB-second. Regional pricing must be checked before deployment.

At those example rates, a continuously running 0.5-vCPU/1-GB container is roughly $18/month in raw compute before networking and other cloud services. This is a planning calculation from published rates, not a cloud quotation.

Kubernetes becomes reasonable later when Vein Health has sustained traffic, multiple independently scalable services, dedicated GPU pools, sophisticated enterprise networking, or a platform team capable of operating it.

Infrastructure options and planning budget

Component	Lean MVP option	Pilot planning range / month	Recommendation
Frontend hosting/CDN	Static/serverless	$0-$20	Use
API containers	Serverless containers	$20-$60	Use
Worker/scheduler	Scale-to-zero container/jobs	$5-$30	Use
PostgreSQL	Serverless managed Postgres	$10-$75	Use
Vector search	pgvector	Incremental DB cost	Use
Object storage	S3-compatible	$5-$25	Use
Queue/cache	Managed queue/Redis	$0-$30	Use
Monitoring	OpenTelemetry + managed backend	$0-$50	Use
AI models	Managed + selective self-host	$15-$50 / 1K MAU	Use
Kubernetes	Managed K8s	Much higher operational baseline	Defer
Dedicated vector database	Managed vector service	Additional service fee	Defer

Component	Lean MVP option	Pilot planning range / month	Recommendation
Always-on GPU	Dedicated GPU	Materially higher	Avoid initially

As a lower-bound Postgres reference, Neon currently lists usage-based Launch compute at $0.106/CU-hour, $0.35/GB-month storage and scale-to-zero; its own example puts a low-use 0.25-CU workload running nine hours per day at about $7.16/month in compute before storage/egress. Its Scale tier adds features including private networking and additional security/compliance capabilities at higher rates.

The provider should not be selected solely from those prices. Health-data contractual requirements, geographic processing, encryption/key-management options, support, backups and regulatory commitments must be verified before production.

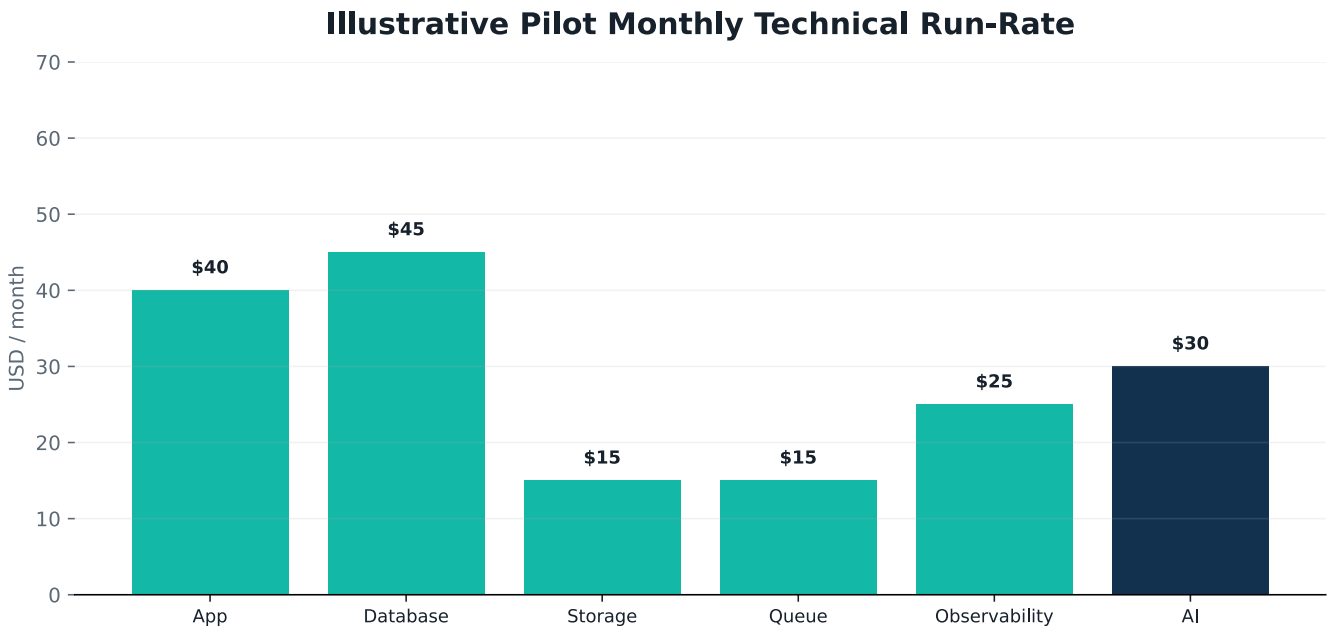
Monthly pilot run rate

A practical 1,000-MAU engineering budget is:

Cost center	Lean	Safer production pilot
Web/API compute	$20	$60
Database	$15	$75
Object storage/backups	$5	$25
Queue/scheduler	$5	$25
Observability	$0	$50
AI inference	$15	$50
Misc. network/email	$10	$30
Approximate total	**$70**	**$315**

Therefore an appropriate planning statement is:

> **Expect roughly $50-$300 per month for the technical pilot platform before substantial traffic, SMS volume, support contracts, legal work, regulatory work or enterprise-grade compliance commitments.**



Illustrative pilot monthly technical run-rate.

These are budgeting midpoints, not vendor quotes.

Security architecture

Because this platform handles health information, security requirements are product requirements, not infrastructure housekeeping. The Nigeria Data Protection Commission has explicitly described health records as sensitive and has worked with health-sector institutions around compliance with the Nigeria Data Protection Act 2023.

Required MVP controls include:

Domain	MVP requirement
Authentication	Managed OIDC; secure account recovery; MFA capability
Authorization	Patient ownership checks on every request
Tenant isolation	`tenant_id` / patient scope enforced server-side
Encryption transit	TLS
Encryption storage	Database/object-store encryption
File access	Short-lived signed URLs
Secrets	Managed secret store
Logging	Structured logs; PHI scrubbers
Auditing	Append-only access/security events
AI requests	Minimum necessary patient context
Provider transmission	Only approved model providers under acceptable data terms

Domain	MVP requirement
Backups	Automated encrypted backups + restore test
Deletion	Defined retention/deletion workflow
Consent	Versioned, auditable consent
Exports	Explicit patient authorization
Dependency security	Automated vulnerability/secret scanning
Admin access	MFA + least privilege
Incident response	Documented security/event procedure

Do not place raw patient data in logs. Use opaque patient/document identifiers, and retrieve details through authorized application paths only.

Do not place entire medical documents on queue messages. Queue jobs should carry an opaque `document_id`, allowing authorized workers to retrieve data from encrypted storage.

Do not send the patient's entire health history to the model on every turn. Retrieve the minimum evidence required for that task.

That improves both privacy **and** token cost.

Regulatory assumptions

The current document establishes Nigeria as the company location and Africa → Global as market direction, but the exact first regulatory jurisdiction, clinical claims, data residency, medical-device classification and cross-border transfer model remain incomplete.

For planning purposes:

Specified: Nigerian company context; health information processing; patient-facing product.

Not yet specified: final launch jurisdiction(s), whether all data stays in a specific region, whether a regulated clinical decision-support function will be claimed, formal device classification, clinical governance structure, retention policy, minimum patient age, child/guardian workflow.

These items must be resolved before general production launch.

Delivery Lifecycle, Team and Budget

A health product should use gated development rather than “build every feature, then test safety at the end.”

The recommended lifecycle is:

```
Define risk
  ↓
Define requirements
  ↓
Build structured data foundation
  ↓
Build AI pipeline
  ↓
Evaluate against gold data
  ↓
Clinician review
  ↓
Red-team
  ↓
Controlled pilot
  ↓
Measure
  ↓
Release / revise
```

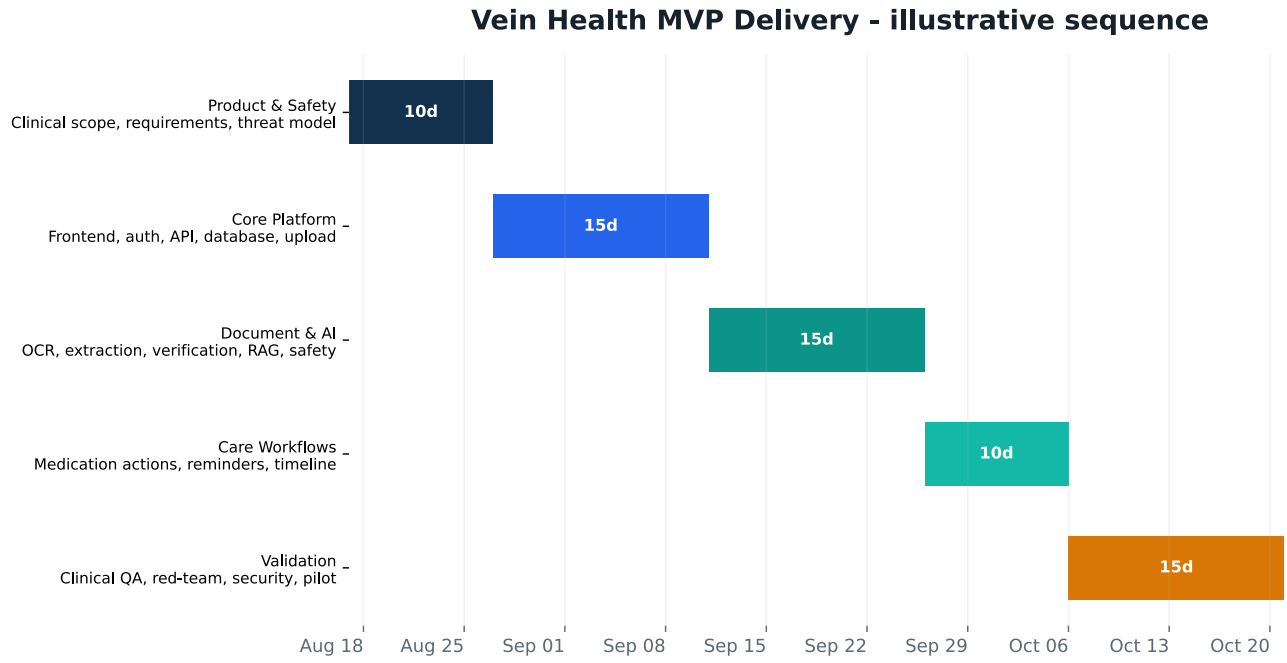
Delivery phases

Phase	Weeks	Deliverables	Main release gate
Clinical/product foundation	1-2	Final MVP scope, safety policies, schemas, threat model, clinical gold-set design	Product claims/safety boundaries approved
Core platform	2-3	Auth, consent, React shell, Postgres, uploads, audit, object storage	Secure upload → patient linkage works
Document + AI pipeline	3	OCR, extraction, confidence, verification UI, RAG, AI gateway, rules engine	Medication critical-field safety passes
Care actions + timeline	2	Medication schedules, reminders, symptoms, side effects, timeline, summaries	Complete Upload → Act workflow
Validation + controlled pilot	2-3	Clinician evaluation, red-team, security, load, monitoring, pilot	Formal release gates pass
Total	**10-13 weeks**	**Patient-facing MVP**	**Pilot-ready**

These timelines assume scope discipline. Adding caregiver permissions, clinic accounts, messaging, diagnosis, insurance, full FHIR integration or complex therapeutic recommendations would materially extend them.

Calendar illustration

Starting from August 17, 2026:



Illustrative MVP delivery sequence starting August 17, 2026.

Parallel work can reduce calendar time, but clinical safety validation should not be shortened merely to hit an arbitrary launch date.

Recommended team

A lean but credible team is:

Role	Allocation	Responsibility
Founder/Product Lead	0.5-1 FTE	Requirements, prioritization, user research
Frontend engineer	1 FTE	React + TypeScript + TanStack patient experience
Backend/AI engineer	1 FTE	FastAPI, PostgreSQL, documents, AI gateway, RAG, workers, safety
Product/UI designer	0.25-0.5 FTE	Mobile-first workflows
Clinician advisor	4-8 hr/week	Safety criteria and evaluation
Security/privacy specialist	Milestone-based	Threat model, launch review
QA	Shared initially	Automated + exploratory testing

A single strong engineer can build a prototype, but a one-engineer implementation pushes a realistic schedule closer to **12-16 weeks** and increases key-person and review risk.

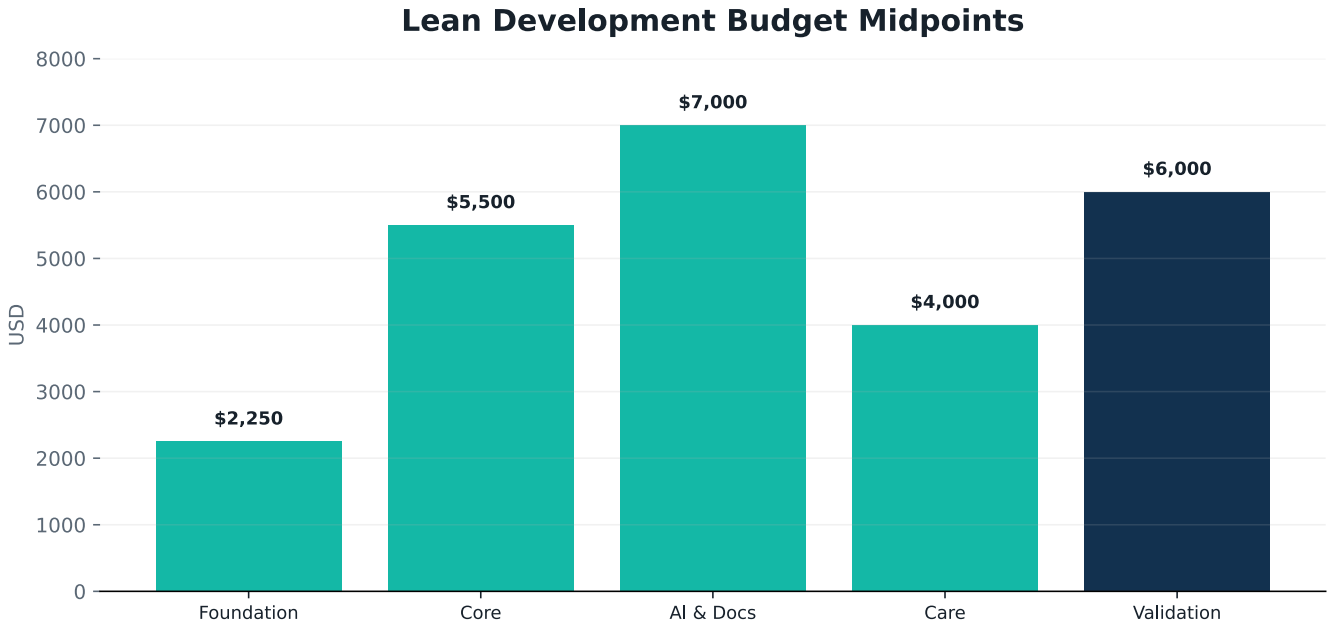
Development budget

The following is a planning budget rather than a Nigerian or global salary benchmark. It assumes lean contract/founding-stage engineering rather than a fully staffed enterprise team.

Phase	Development labor	Infra + model experimentation
Product/clinical foundation	$1,500-$3,000	$25-$100
Core platform	$4,000-$7,000	$50-$150
AI/document pipeline	$5,000-$9,000	$100-$300
Care workflows	$3,000-$5,000	$75-$250
Validation/pilot	$4,000-$8,000	$150-$500
Total	**$17,500-$32,000**	**$400-$1,300**

This excludes legal/regulatory counsel, clinical-study expenses, insurance, company salaries, marketing and large-scale patient support.

Where founders perform a significant share of frontend/backend work themselves, the **cash** development requirement can reasonably fall into the **$6,000-$14,000** range for external design, clinical, security and specialist engineering assistance, but that does not mean the work itself costs less; it means founder labor is not being represented as cash expenditure.



Lean development budget midpoint illustration.

The midpoint represented above is approximately **$24,750** in engineering/product delivery spend.

What should actually be paid for first

The first paid services should be kept deliberately small:

Spend	Early-stage priority
Domain + transactional email	Immediate
Frontend hosting	Minimal/free tier initially
Managed database	Free/developer → paid before PHI production
Object storage	Usage based
Error monitoring	Free/small tier initially
AI API	Usage based only
GPU	Scale-to-zero only when testing medical checker
SMS	Defer or make optional
Kubernetes	Do not purchase
Dedicated vector DB	Do not purchase
Dedicated GPU instance	Do not purchase
Enterprise observability	Do not purchase initially

The technical architecture should therefore be capable of being **enterprise-shaped without carrying enterprise infrastructure costs on day one**.

Quality, Safety, Failure Modes and Release Gates

A healthcare AI application should assume that models will eventually fail. WHO specifically identifies risks from false, inaccurate, biased and incomplete outputs as well as automation bias and cybersecurity threats.

The architectural objective is consequently not:

> “Choose a perfect model.”

It is:

> **Ensure that a model failure rarely becomes a patient-safety failure.**

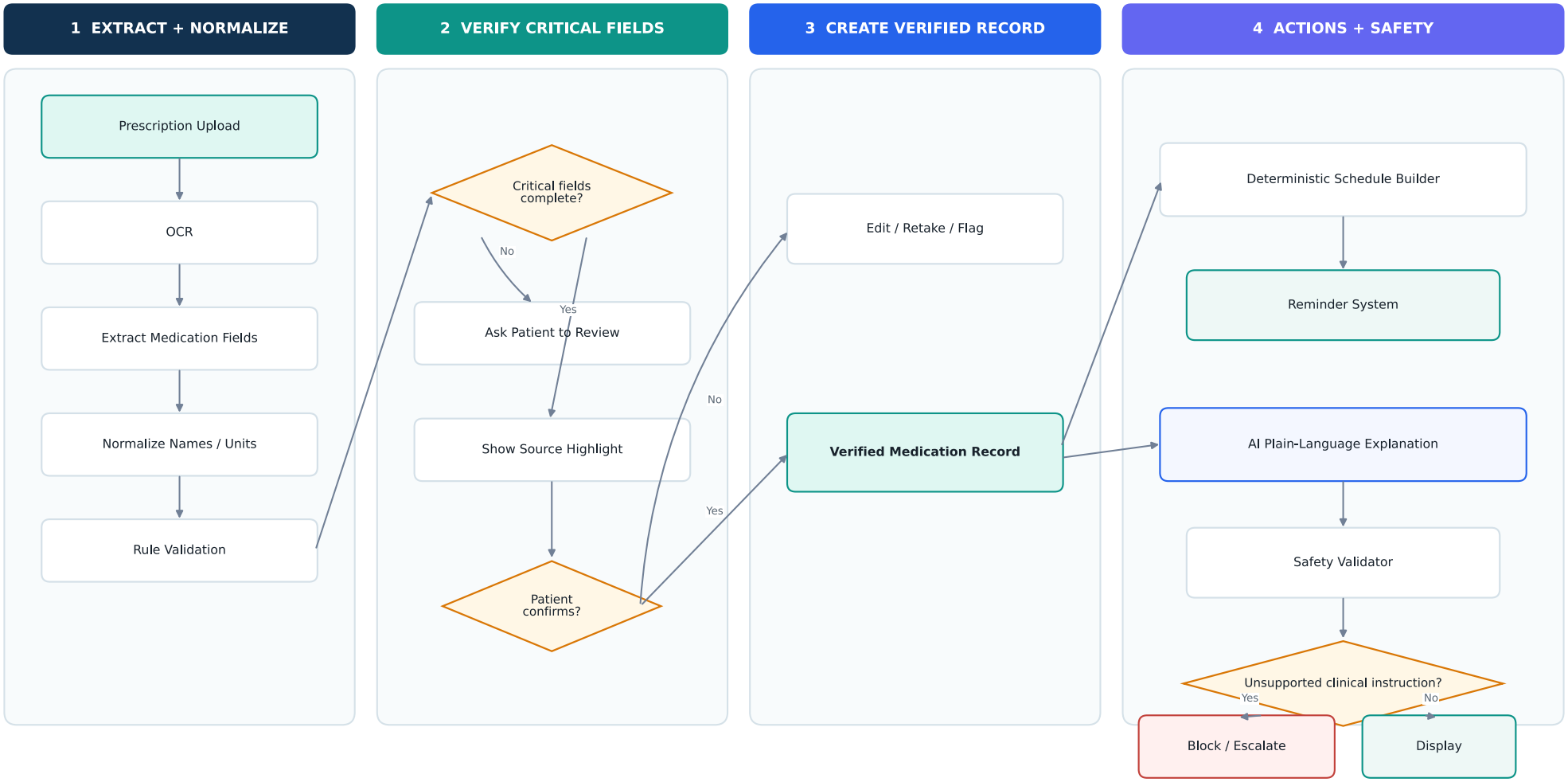
Failure-mode analysis

Failure mode	Potential consequence	Required mitigation
OCR reads `0.5 mg` as `5 mg`	Medication error	Unit/value validators + image highlight + patient confirmation
OCR reads `mcg` as `mg`	Potentially severe dose error	Critical unit rules; never auto-accept
Model invents dosage	Medication error	Dosages can only originate from verified structured fields
Model infers missing duration	Patient over/under-use	Display “duration not provided”
Model contradicts prescription	Confusion	Prescription source wins; conflict shown
Two prescriptions conflict	Wrong medication action	No automatic reconciliation
General model sounds overly confident	Automation bias	Evidence requirement + calibrated uncertainty
Medical checker agrees with wrong primary answer	False reassurance	Do not rely on model agreement alone; rules/ evidence/clinician tests
Emergency symptom missed	Delayed care	High-sensitivity rules + classifier + conservative escalation
False emergency escalation	User inconvenience/ anxiety	Tune specificity after sensitivity requirements met

Failure mode	Potential consequence	Required mitigation
User requests diagnosis	Scope creep into high-risk behavior	Task policy + safe redirection
User requests dosage adjustment	Treatment decision	Decline; clinician/pharmacist escalation
Prompt injection appears inside uploaded PDF	AI manipulation	Treat document as data, never instructions
Wrong patient's history retrieved	Privacy/clinical harm	Patient/tenant scoping at query and authorization layers
Stale medical guidance retrieved	Wrong explanation	Versioned, reviewed, jurisdiction-tagged knowledge
Model provider outage	Feature unavailable	Provider abstraction + degraded deterministic workflow
Provider increases prices	Cost spike	Model gateway + replaceable model adapters
Excessive context usage	Runaway cost	Retrieval caps, token budgets, caching
Model version changes silently	Regression	Pin version; rerun evaluation suite
Sensitive data appears in logs	Privacy breach	Logging allowlist/redaction
User deletes account but backups persist indefinitely	Privacy risk	Defined retention/deletion schedule

Medication-error prevention architecture

Medication should be the most constrained workflow in the MVP:



Medication-error prevention architecture.

The AI model should never be the source of truth for `dose`, `frequency`, `route`, `duration`, or prescription dates.

It may explain those fields once they exist.

That distinction is the single most important architectural decision for minimizing medication errors.

Clinical QA program

The QA program should contain several independent layers.

Document gold set. Build a clinician-reviewed corpus containing prescriptions, laboratory forms, discharge summaries and consultation notes. Include poor photography, shadows, rotated images, low contrast, handwriting where feasible, mixed capitalization, common Nigerian medication brands, decimals, `mg`/`mcg`, once-daily/BD/TDS-like notation, dates and duplicate documents.

Synthetic adversarial set. Programmatically generate thousands of combinations of medication names, dosages, units, frequencies, timestamps and image degradations. Synthetic cases are particularly useful for rare but dangerous formatting combinations without requiring real patient information.

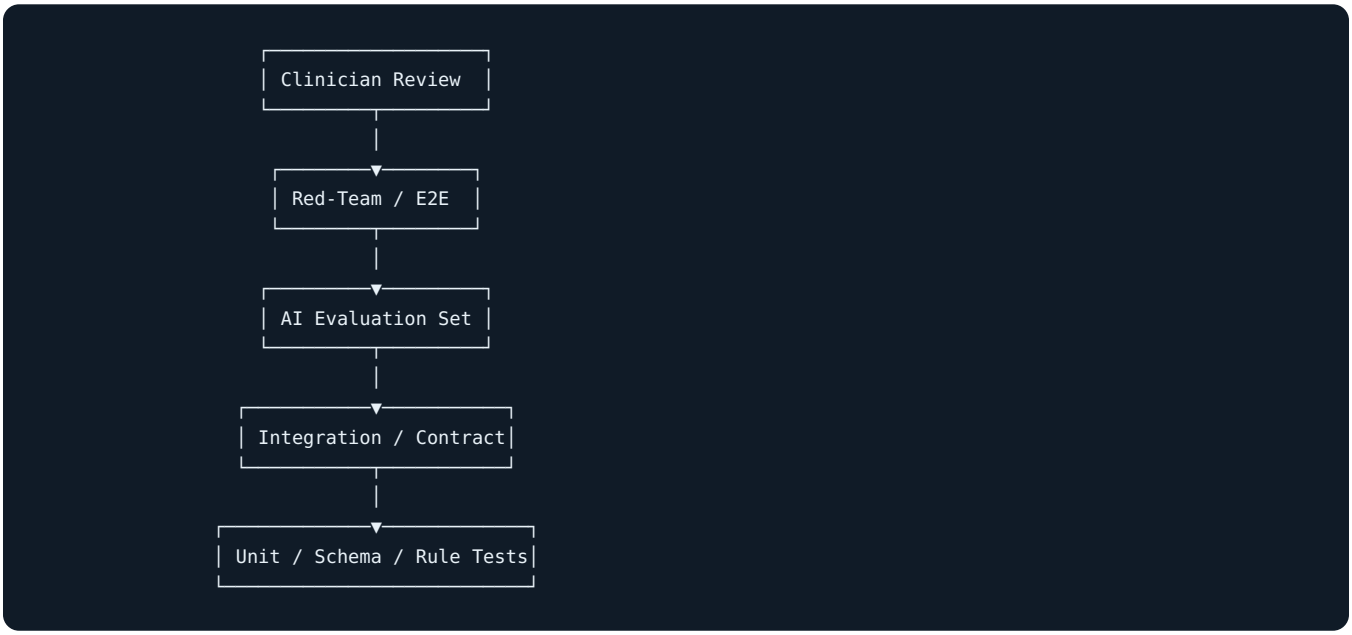
Explanation gold set. For each approved document scenario, clinicians define what the system may safely say, what evidence supports it, what must not be inferred and which cases require escalation.

Risk/red-flag set. Clinicians label must-catch escalation cases. The most safety-critical canonical examples should have **zero false negatives in the release suite**. Broader test-set sensitivity targets can then be defined statistically after sufficient labeled samples exist.

Prompt-injection suite. Example document text should intentionally contain instructions such as “ignore previous instructions,” fake system messages, encoded text and contradictory directives. Documents are untrusted source material and should never gain instruction-level authority.

Security suite. Include broken-object-level authorization, cross-patient identifiers, expired signed links, upload attacks, malformed PDFs, oversized payloads, API rate attacks, CSRF/session abuse where applicable, secrets exposure and backup recovery.

Automated test pyramid



Every model, prompt, RAG-corpus or safety-rule change should run the same regression suite.

AI change-control policy

Every AI execution record should capture:

```
{
  "task": "EXPLAIN_PRESCRIPTION",
  "modelFamily": "...",
  "modelVersion": "...",
  "promptVersion": "prescription-explain-v7",
  "retrievalVersion": "clinical-corpus-2026-08",
  "sourceDocumentIds": ["..."],
  "evidenceIds": ["..."],
  "riskFlags": [],
  "safetyRuleVersion": "safety-v11",
  "inputTokens": 2381,
  "outputTokens": 412,
  "latencyMs": 1840,
  "estimatedCostUsd": 0.0006,
  "createdAt": "..."
}
```

The audit record should preserve input/output data appropriate to privacy policy, but should **not** attempt to preserve or expose hidden internal reasoning traces. What matters operationally is the source evidence, externally visible response, model version, prompt version, safety decision and final outcome.

FHIR's Provenance resource exists specifically to capture context about how information came to be in its current state, while AuditEvent captures security/audit events; those concepts provide a useful design reference even before Vein Health implements external FHIR interfaces.

Pre-launch gates

The MVP should not enter uncontrolled public release until all of the following are true:

Gate	Required state
Clinical scope	Signed off by product + clinical advisor
Data protection	Data-flow/privacy review complete
Regulatory position	Launch claims/classification reviewed
Medication provenance	100% enforced by application logic
Critical medication verification	Mandatory before schedules/reminders
Clinical gold set	All severe-harm cases pass
Red-flag canonical suite	No must-catch misses
Prompt injection	Tested against document attacks
Cross-patient authorization	No known access failures
Backup restore	Tested
Account deletion	Tested
AI cost limits	Per-user/request budgets implemented
Rate limits	Implemented
Model versioning	Pinned
Model rollback	Operational
Safety kill switch	Operational
Monitoring	Error, latency, cost and safety dashboards active
Pilot support	Escalation/contact workflow defined

Final MVP recommendation

The recommended first-release stack is therefore:

```
PATIENT EXPERIENCE
React 19.2
TypeScript
Vite
TanStack Router
TanStack Query
TanStack Form
TanStack Table
Zod
Radix UI / shadcn
Tailwind
Playwright + Vitest

                ↓

APPLICATION
Python
FastAPI
Pydantic
REST + OpenAPI
SQLAlchemy
Alembic
Background workers / jobs

                ↓

DATA
PostgreSQL
pgvector
Encrypted object storage
Managed queue
FHIR-inspired internal schema

                ↓

DOCUMENT PIPELINE
PaddleOCR
PaddleOCR-VL 1.5 0.9B
Structured extraction
Confidence scoring
Patient verification

                ↓

AI GATEWAY
Rules-first risk layer
GTE Multilingual embeddings
RAG
Qwen3.5 9B ↔ Mistral Small 4 evaluation
Selective 4B medical checker
Selective 27B fallback

                ↓

SAFETY
Source-grounded medication fields
Deterministic schedule creation
No invented dosage
Evidence enforcement
Red-flag rules
Audit trail
Human escalation
```

The model strategy should therefore **not** be “buy the strongest medical model available.”

It should be:

> **Use the smallest model that passes Vein Health's task-specific clinical evaluation, use deterministic software whenever a deterministic answer exists, use source evidence wherever evidence exists, and pay for stronger intelligence only when the risk router demonstrates that it is necessary.**

That architecture directly supports the original mission - turning fragmented post-consultation health information into an organized, understandable and actionable patient journey - while keeping the product focused on assistance rather than autonomous medical practice. It also aligns with WHO's recommendation that health AI systems be built for narrowly defined tasks with explicit attention to reliability, privacy, evaluation and oversight.

The resulting MVP is technically credible, substantially cheaper than a two-large-model architecture, and realistically achievable in approximately **10-13 weeks**, with ongoing pilot infrastructure in approximately the **low hundreds of dollars per month rather than thousands**, provided Vein Health maintains the narrow scope and implements the routing, verification and deterministic safety controls described above.